

☁ CLOUDASTICK SYSTEMS — SALESFORCE PARTNER

ABC Pharmacy Salesforce Solution

Technical Documentation & Architecture Decision Record

PROJECT

ABC Pharmacy Order Management System

API VERSION

67.0 — Winter '26

PREPARED BY

Abdelrhman Ehab Abdelkhalek

DATE

August 2026

ORG INSTANCE

orgfarm-7e74891727-dev-ed

ASSESSMENT

Salesforce Entry — Cloudastick Systems

Table of Contents

Solution Overview

Data Model & Design Decisions

Entity Relationship Diagram

Objects Reference

Key Design Decisions & Rationale

Apex Components

Salesforce Features Used

API Reference

Org Access

1. Solution Overview

ABC Pharmacy required a Salesforce environment to manage its product catalog, customer registry, and order lifecycle — integrated with an existing **Ionic web application** and a **warehouse interface** for fulfilment staff.

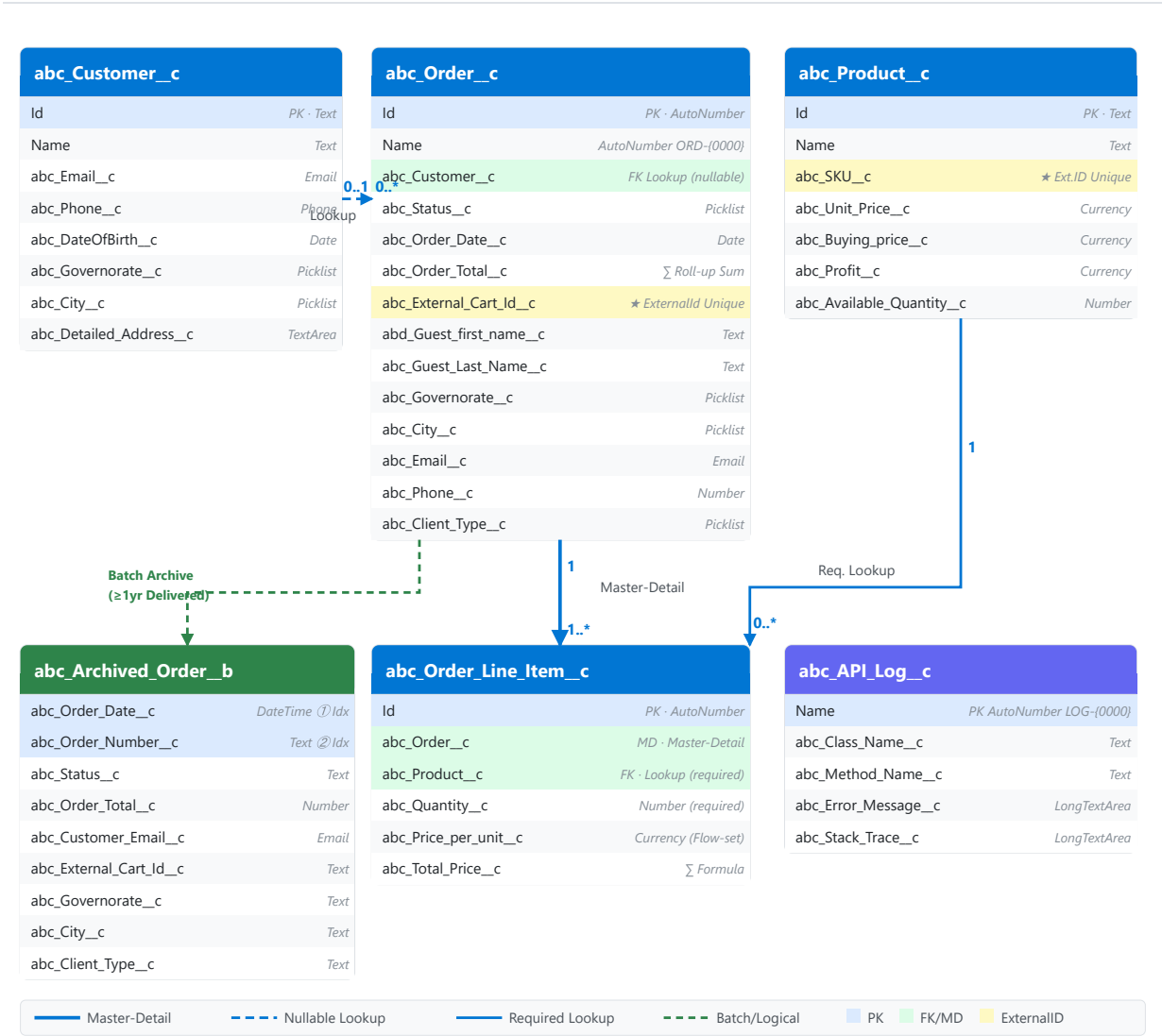
Architecture at a Glance

Layer	Component	Audience
Data	5 Custom Objects + 1 Big Object + 1 Log Object	All users
Automation	Record-Triggered Flow (price freeze) · Roll-up Summary (order total)	Internal
Integration	REST API — GET /products · POST /cart	Ionic web app
UI — Internal	Lightning App + Custom Lightning Record Pages	Admin, Pharmacist
UI — Warehouse	Visualforce Page (abc_WarehouseOrders)	Warehouse user
Archival	Batch Apex → Big Object	Scheduled / Admin
Observability	abc_API_Log__c structured error records	Admin
Security	Custom Profiles (pharmacist, warehouse) + FLS	All users

Convention: All custom metadata follows the `abc_` API name prefix as required.

2. Data Model & Design Decisions

2.1 Entity Relationship Diagram



2.2 Objects Reference

Object	Type	Purpose
abc_Customer__c	Custom	Registered pharmacy customer profiles

Object	Type	Purpose
<code>abc_Product__c</code>	Custom	Product catalog with pricing and live stock levels
<code>abc_Order__c</code>	Custom	Orders for both registered and guest users
<code>abc_Order_Line_Item__c</code>	Custom	Ordered items — Master-Detail to Order, Lookup to Product
<code>abc_Archived_Order__b</code>	Big Object	Immutable archive of delivered orders older than 1 year
<code>abc_API_Log__c</code>	Custom	Queryable error log written by REST API classes on exception

2.3 Key Design Decisions & Rationale

Decision 1 — Custom Objects vs. Standard Salesforce Objects

WHY CUSTOM OBJECTS

Standard objects carry implicit CRM meaning (Opportunity = sales pipeline, not a completed order), complex lifecycle rules, and irrelevant fields. Custom objects give a clean pharmacy domain model, avoid polluting standard CRM records, and simplify permissions.

Decision 2 — Nullable Lookup for Customer → Order

WHY NULLABLE

The Ionic app supports **guest checkout**. A required relationship forces creating ghost Customer records for every guest. Guest orders store contact details directly on the Order. SetNull on delete preserves order history without cascade-deleting orders.

TRADE-OFF

Conditional logic when displaying customer details — acceptable given the guest checkout requirement.

Decision 3 — Master-Detail for Line Item → Order

WHY MASTER-DETAIL

Unlocks three features at zero code cost: (1) **Roll-up Summary** for real-time order total, (2) **cascade delete** — deleting an order removes its line items, (3) enforced referential integrity. A Lookup would

require Apex triggers for all three.

Decision 4 — Before-Save Flow for Price Freezing

Approach	Extra DML?	Admin Maintainable?	Retroactive Risk
✓ Before-save Flow	No	Yes	None — frozen on create
After-save Apex Trigger	Yes	No	None
Formula Field	No	Yes	HIGH — always reflects current price

Decision 5 — Big Object for Archiving

Approach	Storage Cost	Scalability	Queryable
✓ Big Object	Included (no limit)	Petabyte-scale	SOQL with index
Custom Object	Against 10GB limit	Limited	Full SOQL

WHY BIG OBJECT

Standard storage (10GB default) would eventually be exhausted. Big Objects are designed for high-volume, append-only historical data at no additional cost. The compound index (Order Date DESC, Order Number ASC) ensures efficient retrieval.

Decision 6 — Custom REST API vs. Standard Salesforce API

WHY CUSTOM REST

The standard API would require Ionic to know internal SF record IDs, make multiple round-trips, and implement idempotency client-side. A custom API accepts SKU codes, creates Order + Line Items atomically, handles idempotency via externalCartId, and returns clean HTTP status codes (201/400/404/422/500).

Decision 7 — without sharing on WarehouseOrdersController

WHY WITHOUT SHARING

Orders are created by the Ionic API or pharmacists — not by the warehouse user. With sharing, the warehouse profile sees only records they own (none). Without sharing is safe because the VF page itself is restricted to the warehouse profile.

Decision 8 — Custom Log Object vs. Debug Logs vs. Platform Events

Approach	Persistent?	Admin Visible?	Queryable?
✓ abc_API_Log__c	Yes	Yes	Yes (SOQL)
Debug Logs	Until purged (2MB)	Dev only	No
Platform Events	72h only	Needs subscriber	Via subscription

3. Apex Components

3.1 REST API Classes

Class	URL / Method	Purpose & Key Detail
abc_ProductAPI	GET /abc/products	Returns products with stock > 0, sorted A-Z. Full try/catch → abc_APILogger → HTTP 500 on error.
abc_CartAPI	POST /abc/cart	Creates Order + Line Items atomically. Accepts SF IDs or SKU codes. Idempotent via externalCartId. Returns 400/404/422 for specific validation failures. Phone field (Number type) strips non-numeric chars before assignment.

3.2 Supporting Classes

Class	Type	Detail
abc_APILogger	public without sharing	Inserts abc_API_Log__c records. Falls back to System.debug if DML fails. without sharing ensures logs are always written.
abc_ArchiveOrdersBatch	Database.Batchable	Delivered orders where Order Date < TODAY-1yr → Database.insertImmediate() to Big Object. Run: Database.executeBatch(new abc_ArchiveOrdersBatch(), 200);
abc_WarehouseOrdersController	public without sharing	Loads all orders + line items in one SOQL sub-query. OrderWrapper resolves icon (⌚ ⌚ 🚚 ✅). Row expansion via client-side JS.

3.3 Test Classes

Class	Covers	Scenarios
abc_ProductAPITest	abc_ProductAPI	In-stock only returned; empty catalog → 200

Class	Covers	Scenarios
		count:0
<code>abc_CartAPITest</code>	<code>abc_CartAPI</code>	Success by SF ID; by SKU; idempotency (200); 400/404/422 error paths; blank productId; invalid customerId
<code>abc_ArchiveOrdersBatchTest</code>	<code>abc_ArchiveOrdersBatch</code>	Old delivered archived; recent excluded; draft excluded; customer email preferred
<code>abc_WarehouseOrdersControllerTest</code>	<code>abc_WarehouseOrdersController</code>	All 4 icon types; line items loaded; total order count

4. Salesforce Features Used

Feature	Component	Why Chosen
Custom Lightning App	abc_PharmaD	Branded navigation surfacing only pharmacy tabs, separate from standard Sales/Service apps.
Custom Objects & Fields	6 objects, 30+ fields	Clean domain model. AutoNumber for Order Name, Roll-up for totals, Formula for line totals, Currency for pricing.
Big Object	abc_Archived_Order__b	Unlimited-scale append-only data. Database.insertImmediate() bypasses standard DML governor limits.
Record-Triggered Flow (before-save)	abc_product_and_line_item_price_match	Freezes price at order time with no extra DML. Admin-maintainable without code deployments.
Roll-Up Summary	abc_Order_Total__c	Real-time order total at zero Apex cost — only possible via the Master-Detail relationship.
REST Apex	abc_ProductAPI, abc_CartAPI	Single atomic request for both Order + Line Items. Business rules enforced server-side. Clean HTTP status codes for Ionic app.
Batch Apex	abc_ArchiveOrdersBatch	Large-scale scheduled archiving within governor limits. Cannot be replaced by Scheduled Flow (Flows can't call Database.insertImmediate).
Visualforce Page	abc_WarehouseOrders	Required by assessment. Server-side icon logic + client-side JS for row expansion without page reloads.
Custom Profiles	pharmacist, warehouse	Least privilege. Warehouse: read-only + VF page. Pharmacist: full order lifecycle.
Lightning Record Pages	4 Flexipages + utility bar	Polished Lightning UI, admin-configurable without code.

Feature	Component	Why Chosen
Global Value Sets	abc_Governorate_values, abc_City_values	Egyptian geography shared across Customer and Order. Single source of truth.
External ID Fields	abc_SKU__c, abc_External_Cart_Id__c	SKU lets Ionic use real-world codes. ExternalCartId enables idempotency. Both are indexed.
Custom WebLinks	Warehouse Orders View buttons	One-click access to VF page from Order record and list view.
Error Log Object	abc_API_Log__c	Persistent, queryable API error trail visible to admins without debug sessions.

5. API Reference

Authentication

OAuth2 Bearer token required. For production, use the **JWT Bearer Flow** with a Connected App.

```
Authorization: Bearer <access_token>
```

GET /services/apexrest/abc/products

```
GET https://<instance-url>/services/apexrest/abc/products
Authorization: Bearer <token>

200 OK: { "success": true, "count": 2, "data": [
  { "id": "...", "name": "Amoxicillin 250mg", "sku": "SKU-AMOX-250",
    "unitPrice": 35.0, "availableQuantity": 100 }
] }
500: { "success": false, "error": "..." }
```

POST /services/apexrest/abc/cart

```
POST https://<instance-url>/services/apexrest/abc/cart
Content-Type: application/json

{ "externalCartId": "CART-001", "customerId": "<SF ID or omit for guest>",
  "guestFirstName": "Ahmed", "guestLastName": "Ali",
  "governorate": "Cairo", "city": "Nasr City",
  "email": "ahmed@example.com", "phone": "01001234567",
  "detailedAddress": "123 Main St",
  "items": [{ "productId": "SKU-PARA-500", "quantity": 2 }] }
```

Status	Condition	Body
201	Order created	{ "success": true, "orderId": "...", "orderNumber": "ORD-0001" }
200	Cart already exists (idempotent)	{ "success": true, "message": "Cart already exists.", "orderId": "..." }

Status	Condition	Body
400	Validation failure	<code>{ "success": false, "error": "... " }</code>
404	Product not found	<code>{ "success": false, "error": "Product not found: SKU-X" }</code>
422	Insufficient stock	<code>{ "success": false, "error": "Insufficient stock... (available: X, requested: Y)" }</code>
500	Server error (logged to abc_API_Log_c)	<code>{ "success": false, "error": "... " }</code>

6. Org Access

Instance URL

```
https://orgfarm-7e74891727-dev-ed.develop.my.salesforce.com
```

Reviewer Account (System Administrator)

Username	assessments@cloudastick.com.pharmaD
Email	assessments@cloudastick.com
Profile	System Administrator

Note: Use "Forgot Password" at the instance URL to set the password before logging in.

Warehouse User

Profile	warehouse
Direct URL	/apex/abc_WarehouseOrders
Via Order record	"Warehouse Orders View" button on any Order record
Via list view	"Warehouse Orders View" in Orders list view actions